NEXFLIS 2.0

Reporte final del proyecto
Sistema de Recomendación de Películas
Recuperación de la Información

Alumno: Adolfo Huerta Martínez
Matrícula: 202262175

Mayo 2026

Índice

1. Introducción **3**

2. Arquitectura del sistema **3**

2.1. Estructura de carpetas 3

2.2. Stack tecnológico 3

3. Modelos de RI implementados **4**

3.1. Modelo booleano 4

3.2. Modelo probabilístico con BM25 4

3.3. Modelo vectorial por dimensiones 4

4. Técnicas IR avanzadas **5**

4.1. Jaccard ponderado con IDF 5

4.2. Similitud semántica por TF-IDF coseno 5

5. Matriz documento–término **5**

6. Motor de descubrimiento (modo “Algo Diferente”) **6**

7. Autenticación y seguridad **6**

7.1. Autenticación con sesiones Flask 6

7.2. Rate limiting 6

7.3. Protección CSRF 6

7.4. Sistema de pagos simulado con cifrado bancario 7
7.4.1. La llave secreta de cifrado 7
7.4.2. Herramientas de codificación y decodificación 7
7.4.3. Flujo completo de una transacción 8
7.5. Exportación segura del historial de usuario 8
7.5.1. Módulo criptográfico compartido (`crypto_utils.py`) 8
7.5.2. Estructura del XML cifrado 8
7.5.3. Cómo se calcula y verifica la firma HMAC 9
7.5.4. Flujo completo de exportación y verificación 10
7.5.5. Generación del PDF del historial 10
7.6. Eliminación de cuenta 11
7.7. Restricción de contenido por edad 11

8. Conclusiones **11**

1. Introducción

En este proyecto se desarrolló NEXFLIS, pensado en explicar completamente como es que funcionan los recomendadores integrados en aplicaciones de renta de películas (streaming), incorporando explícitamente tres enfoques de Recuperación de la Información: modelo booleano para búsquedas, modelo vectorial por dimensiones para similitud explicable y modelo probabilístico con BM25.

El sistema sigue la siguiente lógica: primero obtiene candidatos con un modelo de recuperación (booleanas y probabilísticas, dependiendo de la busqueda) y después aplica una función de evaluación ponderada para producir recomendaciones interpretables para el usuario.

Adicionalmente, la implementación incorpora características de sistema de producción: autenticación segura con sesiones HttpOnly, restricción de contenido por edad, frontend React interactivo, logging, generación de contenido cifrado, y pago de membresía con combrobablee descargable.

2. Arquitectura del sistema

La arquitectura sigue una separación modular de la siguiente manera.

2.1. Estructura de carpetas

```
nexflis/
  app.py                  # Punto de entrada Flask
  core/                   # Motor de recomendacion y logica RI
    motor_recomendacion.py
    motor_explicacion.py
    gestor_dimensiones.py
    logger.py
  models/                 # Modelos del backend
  routes/                 # Blueprints Flask (auth, busqueda)
  services/               # Integraciones de servicios(TMDB, DB)
  templates/              # HTML Jinja2
  frontend/               # Aplicacion React
    src/
      api/                # Clientes HTTP (endpoints)
      components/         # Componentes UI reutilizables
      pages/              # HomePage, AuthPage
```

2.2. Stack tecnológico

- **Backend:** Python 3.13 + Flask 3
- **Fuente de datos:** API de TMDB (The Movie Database).
- **DB:** PostgreSQL (caché TMDB, usuarios, historial, clasificación de edad).

- **Frontend:** React 18 + Vite + axios.
- **Autenticación:** bcrypt (hashing) + Flask session (cookie HttpOnly) + PyJWT.
- **Seguridad:** Flask-Limiter (rate limiting) + Flask-WTF (CSRF) + headers de seguridad HTTP.

3. Modelos de RI implementados

3.1. Modelo booleano

Se implementó un modelo booleano aplicado a búsqueda por dimensiones en el endpoint `POST /api/hero/search`. El usuario puede combinar criterios de *género*, *keyword*, *actor*, *director* y *colección* bajo tres modos lógicos:

- **AND:** intersección de restricciones – solo películas que cumplan *todos* los filtros activos.
- **OR:** unión de grupos de resultados – películas que cumplan al menos uno.
- **NOT:** exclusión de resultados que cumplan los filtros seleccionados.

3.2. Modelo probabilístico con BM25

Se implementó ranking probabilístico mediante BM25 para reordenar candidatos según su relevancia textual frente al historial del usuario. Se usan términos de título, sinopsis, géneros y keywords para construir:

- consulta q derivada del historial reciente,
- documentos d_i como películas candidatas,
- score BM25 por candidato.

3.3. Modelo vectorial por dimensiones

Se implementó una representación vectorial por dimensiones semánticas y de comportamiento. Cada película candidata se evalúa contra el perfil del usuario en componentes normalizados y ponderados.

La matriz de recomendación usa 8 variables:

- R_i: Reparto (15 %) – similitud de actores
- S_i: Saga/colección (10 %) – pertenencia a franquicia
- G_i: Géneros (15 %) – coincidencia de géneros
- D_i: Director (5 %) – mismo director

- M_i: Semántica (20 %) – keywords/sinopsis/título
- A_i: Afinidad temática (10 %) – colección e intención
- U_r: Reacción del usuario (15 %) – historial de reacciones
- U_t: Tiempo visto (10 %) – duración visualizada

Función de evaluación de similitud:

$$f(d_i) = 0.15R_i + 0.10S_i + 0.15G_i + 0.05D_i + 0.20M_i + 0.10A_i + 0.15U_r + 0.10U_t$$

Donde M_i es la señal semántica (keywords/sinopsis/título) y A_i es afinidad temática (franquicia, colección y términos de intención).

4. Técnicas IR avanzadas

4.1. Jaccard ponderado con IDF

Para la comparación de conjuntos de términos (géneros, keywords, reparto) se implementó Jaccard y su variante ponderada con IDF para reducir el peso de términos demasiado comunes.

4.2. Similitud semántica por TF-IDF coseno

La dimensión semántica (M_i) se calcula comparando vectores TF-IDF de sinopsis y keywords. Se usa el coseno como medida de similitud.

5. Matriz documento–término

Se implementó una matriz documento–término práctica donde:

- cada **documento** es una película candidata,
- la **consulta** es el perfil de historial reciente del usuario,
- cada **dimensión/término** representa una señal de similitud o comportamiento,
- cada celda $w_{i,j}$ es la similitud ponderada de la dimensión j para la película i.

El motor de recomendación implementa los métodos modo “Similar” y modo “Algo Diferente”, ambos producen un diccionario con el score por dimensión y el score final.

6. Motor de descubrimiento (modo "Algo Diferente")

Se implementó un segundo modelo para explorar contenido diferente cuando la reacción del usuario es negativa. Este modelo prioriza novedad pero conserva control de calidad y contexto temporal, usando las dimensiones de diferencia seleccionadas por el usuario mediante `DiffModal`.

$$f_{desc}(d_i) = 0.30 \cdot DifGen_i + 0.30 \cdot DifRep_i + 0.20 \cdot DifDir_i + 0.10 \cdot Calidad_i + 0.10 \cdot Tiempo_i$$

El usuario elige qué dimensiones quiere que sean "diferentes" (géneros, reparto, director, semántica, época), y el motor penaliza la similitud en esas dimensiones y la premia como novedad.

7. Autenticación y seguridad

7.1. Autenticación con sesiones Flask

Se implementó autenticación basada en cookies de sesión Flask (HttpOnly, SameSite=Lax, Secure en producción) como mecanismo principal. JWT Bearer se mantiene como mecanismo secundario para clientes API. El middleware (`routes/jwt_middleware.py`) intenta Bearer primero, luego cae a la sesión Flask:

```
def _resolve_usuario():
    auth_header = request.headers.get('Authorization', '')
    if auth_header.startswith('Bearer␣'):
        return _auth_service.verify_token(auth_header[7:])
    return _auth_service.get_usuario_actual(session)
```

7.2. Rate limiting

Se implementó con Flask-Limiter para proteger endpoints de autenticación contra fuerza bruta:

- `POST /login`: 10 solicitudes por minuto por IP.
- `POST /register`: 5 solicitudes por minuto por IP.

7.3. Protección CSRF

Se implementó con Flask-WTF `CSRFProtect` en formularios HTML. Los endpoints `/api/*` están exentos (protegidos por SameSite=Lax + JSON content-type). Se inyecta el token mediante campo oculto en cada formulario POST.

7.4. Sistema de pagos simulado con cifrado bancario

Se implementó un módulo de pagos educativo que simula una transacción de suscripción Premium aplicando los mismos patrones criptográficos que utiliza la banca electrónica (estándar PCI-DSS).

7.4.1. La llave secreta de cifrado

La llave AES utilizada para cifrar los datos de cada transacción se deriva directamente de la variable de entorno `FLASK_SECRET_KEY`, definida en el archivo `.env` del servidor.

```
import hashlib, os
from base64 import b64encode
from cryptography.fernet import Fernet

secret = os.getenv('FLASK_SECRET_KEY')   # clave maestra del
    servidor
key_bytes = hashlib.sha256(secret.encode()).digest()  # 32 bytes
    fijos
fernet_key = b64encode(key_bytes)        # formato base64 URL-
    safe requerido por Fernet
cipher = Fernet(fernet_key)
```

Por qué SHA-256 para derivar la llave: Fernet requiere exactamente 32 bytes en formato base64 URL-safe. La llave maestra del servidor puede ser una cadena de longitud arbitraria, por lo que se aplica SHA-256 para obtener siempre 32 bytes deterministas. Esto garantiza que la misma `FLASK_SECRET_KEY` produzca siempre la misma llave AES, lo que permite descifrar transacciones antiguas sin almacenar la llave de cifrado en la base de datos.

7.4.2. Herramientas de codificación y decodificación

Cifrado AES-256 (confidencialidad):

Se utiliza la librería `cryptography` de Python, concretamente `Fernet`, que implementa AES-128-CBC con PKCS7 padding más autenticación interna. El flujo completo es:

Fernet garantiza que un token alterado (un solo bit cambiado) **no puede descifarse**: lanza `InvalidToken` antes de devolver ningún dato, lo que impide ataques de texto cifrado elegido.

Firma HMAC-SHA256 (integridad):

Se utiliza el módulo `hmac` de la biblioteca estándar de Python:

`hmac.compare_digest` realiza una comparación en tiempo constante para prevenir *timing attacks*: si se usara `==`, un atacante podría medir el tiempo de respuesta para adivinar la firma byte a byte.

7.4.3. Flujo completo de una transacción

1. **Pago simulado:** el usuario ingresa los datos de tarjeta (no almacenados, solo los últimos 4 dígitos).
2. **Generación de transacción:** el servidor crea un ID único (`TXN-XXXXXXXXXXXX`), cifra los datos sensibles con AES-256 y firma el payload cifrado con HMAC-SHA256.
3. **Persistencia:** se almacena en la tabla `transacciones`: `txn_id`, `ultimos_digitos`, `payload_cifrado`, `hmac_firma`, `expires_at`.
4. **Actualización de plan:** en la misma transacción SQL se actualiza `users.plan = 'premium'` y `plan_expires_at` a 30 días desde el momento del pago.
5. **Comprobante PDF:** se genera un PDF con `reportlab` que incluye los datos legibles y el payload cifrado + firma para que el usuario pueda verificarlo independientemente.
6. **Verificación:** el usuario puede pegar el payload cifrado y la firma del PDF en el sistema; el servidor descifra, recalcula el HMAC y confirma la autenticidad sin necesidad de almacenar los datos en texto plano.

7.5. Exportación segura del historial de usuario

Se implementó un módulo de exportación del historial de visualizaciones en dos formatos: PDF legible y XML cifrado.

7.5.1. Módulo criptográfico compartido (`crypto_utils.py`)

Para evitar duplicación de código y garantizar que ambos módulos (`payment_service` e `historial_export_service`) utilicen la misma llave AES y el mismo secret HMAC, se extrajo la lógica de derivación a un módulo centralizado:

Por qué dos secret distintos:

- `FLASK_SECRET_KEY` → deriva la llave AES (confidencialidad). Conocerla permite *descifrar* contenido.
- `JWT_SECRET` → es el secreto HMAC (integridad). Conocerlo permite *forjar* firmas.

Separar ambos secretos sigue el principio de *separación de funciones criptográficas*: comprometer uno no compromete automáticamente el otro.

7.5.2. Estructura del XML cifrado

El XML del historial sigue una estructura de tipo bancario en la que **cada campo sensible se cifra individualmente** con AES-256-Fernet, y la integridad del documento completo se garantiza con una firma HMAC-SHA256 en el atributo raíz:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE historial SYSTEM "nexflis-historial.dtd">
<historial version="1.0" generado="2026-04-22T11:39:00-06:00"
           firma="a3f7c2...">
  <emisor>NEXFLIS</emisor>
  <titular cifrado="AES256">gAAAAA...</titular>
  <registros total="3">
    <registro id="1">
      <pelicula cifrado="AES256">gAAAAA...</pelicula>
      <reaccion cifrado="AES256">gAAAAA...</reaccion>
      <fecha    cifrado="AES256">gAAAAA...</fecha>
      <minutos  cifrado="AES256">gAAAAA...</minutos>
    </registro>
  </registros>
</historial>
```

Por qué cifrado campo por campo (y no el documento completo):

- Permite al descifrador frontend mostrar **cuántos registros** hay y la estructura del documento antes de descifrar, sin revelar el contenido.
- Refleja el modelo bancario real, donde solo los campos sensibles (número de cuenta, monto) van cifrados y los metadatos estructurales (fecha, tipo de operación) pueden ser legibles para el sistema de procesamiento.

7.5.3. Cómo se calcula y verifica la firma HMAC

La firma se calcula **antes de cifrar** los campos, sobre una representación canónica JSON de los datos en texto plano. Esto es fundamental: la firma protege el contenido original, no el token cifrado.

```
# 1. Construir representacion (texto plano, campos ordenados)
datos_para_firma = json.dumps({
    'titular':   usuario['username'],
    'generado':  ahora_iso,
    'registros': [{'id': i, 'pelicula': t, 'reaccion': r,
                   'fecha': f, 'minutos': m} for i, t, r, f, m in
                       ...],
}, ensure_ascii=False, sort_keys=True)

# 2. Firmar con HMAC-SHA256
firma_hex = hmac.new(
    get_hmac_secret(),
    datos_para_firma.encode('utf-8'),
    hashlib.sha256,
).hexdigest()   # 64 caracteres hexadecimales

# 3. Cifrar cada campo por separado
titular_cifrado = get_fernet().encrypt(usuario['username'].encode
   ()).decode()
```

Al verificar, el descifrador realiza el proceso inverso:

1. **Descifra** cada campo con `get_fernet().decrypt(token)`.
2. **Reconstruye** la misma representación canónica JSON con los datos descifrados.
3. **Recalcula** el HMAC sobre esa representación.
4. **Compara** con `hmac.compare_digest` (comparación en tiempo constante, immune a timing attacks).

Si un solo campo fue modificado en el XML (incluso cambiar una letra), el texto descifrado será diferente, la representación canónica será distinta y el HMAC no coincidirá. Si el token Fernet fue alterado directamente, `cryptography.fernet` lanza `InvalidToken` antes de retornar dato alguno.

7.5.4. Flujo completo de exportación y verificación

1. El usuario autenticado solicita `GET /api/usuario/historial/xml` (requiere JWT válido).
2. El servidor consulta `historial_usuarios` en PostgreSQL y obtiene los registros con `viewed_at` en UTC.
3. Las fechas se convierten a hora de México (UTC-6) antes de incluirlas en el XML.
4. Se construye la representación canónica, se firma con HMAC, se cifra campo por campo con AES-256-Fernet y se serializa el XML.
5. El navegador descarga el archivo `historial_{username}.xml`.
6. Para verificar, el usuario abre `HistorialDecryptModal` en el frontend, pega el contenido del XML y hace clic en "Descifrar XML".
7. El frontend envía el XML al endpoint `POST /api/usuario/historial/descifrar`, que descifra, recalcula el HMAC y retorna los registros en JSON.
8. La interfaz muestra la tabla de películas y un badge "Firma válida / Inválida" según el resultado de `hmac.compare_digest`.

7.5.5. Generación del PDF del historial

El PDF se genera con la librería `reportlab` siguiendo el mismo estilo visual que el comprobante de pago: encabezado NEXFLIS, tabla de metadatos del titular y tabla de registros con columnas Película, Reacción, Minutos y Fecha (hora México). A diferencia del XML, el PDF es **totalmente legible**: su propósito es ser un documento de consulta personal, no un comprobante de integridad verificable.

7.6. Eliminación de cuenta

Se implementó la funcionalidad de eliminación permanente de cuenta mediante el endpoint `DELETE /api/users/me`, protegido por JWT y verificación de contraseña.

Flujo de eliminación:

1. El usuario accede a su perfil y hace clic en "Eliminar cuenta" (botón discreto en zona de peligro).
2. Un modal de confirmación advierte que la acción es **permanente e irreversible** y solicita la contraseña actual.
3. El backend verifica la contraseña con bcrypt. Si no coincide, retorna `401` sin realizar ningún cambio.
4. Si la verificación es exitosa, se ejecuta `DELETE FROM users WHERE id = %s`.
5. Gracias a las restricciones `ON DELETE CASCADE` definidas en el esquema de BD, se eliminan automáticamente todos los datos del usuario:

```
-- Tablas con FK CASCADE hacia users
historial_usuarios   (usuario_id) ON DELETE CASCADE
transacciones        (usuario_id) ON DELETE CASCADE
ultima_consulta_usuario (usuario_id) ON DELETE CASCADE
ultima_recomendacion    (usuario_id) ON DELETE CASCADE
```

7.7. Restricción de contenido por edad

El sistema implementa tres niveles de restricción al intentar visualizar una película:

1. **Perfil incompleto** (`edad == null`): se muestra `CompleteProfileModal` para que el usuario registre su edad antes de continuar.
2. **Menor de edad**: si `edad_usuario <clasificacion_edad` se bloquea con `AgeBlockModal`.
3. **Acceso permitido**: si la edad es suficiente, el flujo continúa normalmente.

La clasificación de edad de cada película se persiste en PostgreSQL y se obtiene de TMDB. El filtro también se aplica en el motor de recomendación backend.

8. Conclusiones

En este proyecto se implementó NEXFLIS, un recomendador híbrido alineado con los modelos centrales de Recuperación de la Información:

- **Booleano:** búsqueda por facetas con lógica AND/OR/NOT sobre metadatos de películas.

- **Probabilístico:** ranking textual con BM25 mezclado con señal base de TMDB.
- **Vectorial/dimensional:** función de evaluación ponderada con 8 dimensiones, interpretable y explicable para el usuario.

El resultado es un sistema funcional, con características de producción: recupera candidatos con lógica de RI, los rankea con señales probabilísticas, los explica con una matriz de dimensiones, y garantiza seguridad con autenticación por cookie HttpOnly, rate limiting, protección CSRF y logging centralizado.

Adicionalmente, se incorporó un módulo de seguridad avanzado que demuestra los principios criptográficos aplicados a transacciones digitales y exportación de datos personales: cifrado AES-256-Fernet (confidencialidad), firma HMAC-SHA256 (integridad) y derivación de llaves con SHA-256, siguiendo los mismos patrones que utilizan los sistemas de banca electrónica bajo el estándar PCI-DSS.

El módulo de historial exportable extiende estos temas del ámbito de los datos personales: el XML del historial cifra cada campo individualmente, firma el conjunto con HMAC antes de cifrar (integridad del contenido original) y permite al propio usuario verificar la autenticidad del documento descargado desde la interfaz, demostrando el principio de *no repudio* en un contexto educativo.